



Giga Academy Kosovo Program

Project Report

Stream: AI Engineering

Project Title: Salient Object Detection System

Date:

May 7, 2026

Author:

Florjetë Kuka

Content

1. Abstract.....	3
2. Introduction.....	3
3. Purpose of the project.....	4
4. Main Part.....	5
4.1. Dataset and Preprocessing.....	5
4.1.1. Dataset.....	5
4.1.2. Data Preprocessing.....	5
4.1.3. Dataset Splitting.....	6
4.1.4. Data Augmentation.....	6
4.2. Model Architecture.....	7
4.2.1. Baseline Model.....	7
4.2.2. Improved Model.....	7
4.3. Training Procedure.....	8
4.3.1. Loss Function.....	8
4.3.2. Optimizer and Learning Rate.....	8
4.3.3. Checkpointing and Early Stopping.....	9
4.4. Evaluation Metrics.....	9
4.5. Results and Improvements.....	9
5. Conclusion and Future Work.....	10

1. Abstract

This project presents the development of a Salient Object Detection (SOD) system using a custom convolutional neural network implemented in PyTorch. The goal of the project was to create a deep learning pipeline capable of identifying and segmenting the most visually important object in an image.

The model was trained on the MSRA10K dataset using an encoder-decoder CNN architecture built from scratch without pretrained models. Images and masks were resized to 128×128 and processed using normalization and data augmentation techniques. The system was evaluated using IoU, Precision, Recall, and F1-score metrics.

Several improvements were applied during development, including Batch Normalization, threshold tuning, and longer training. The final model achieved an IoU of 0.6470 and an F1-score of 0.7895 on the test set. The project demonstrates a complete computer vision workflow including preprocessing, training, evaluation, and visualization.

2. Introduction

Salient Object Detection (SOD) is a computer vision task focused on identifying and segmenting the most visually important object or region within an image. Unlike traditional object detection systems that attempt to classify and localize all objects in a scene, SOD aims to detect the dominant object that naturally attracts human visual attention.

Salient object detection has applications in image editing, autonomous systems, visual attention modeling, robotics, image compression, and medical imaging.

The objective of this project was to design and implement a complete Salient Object Detection pipeline from scratch using deep learning techniques. The implementation was developed using PyTorch and trained on the MSRA10K dataset. The project includes dataset preprocessing, CNN model development, training and evaluation logic, visualization of predictions, and inference demonstrations.

3. Purpose of the project

The purpose of this project was to design and implement a complete Salient Object Detection (SOD) system using Deep Learning and Computer Vision techniques. The project focused on building an end-to-end pipeline including dataset preprocessing, CNN model implementation, training, evaluation, and visualization of results.

The MSRA10K dataset was selected because it is a widely used benchmark dataset for Salient Object Detection and contains RGB images with corresponding saliency masks.

The project intentionally avoided pretrained models and instead implemented a CNN encoder-decoder architecture from scratch in PyTorch. Several improvements were later applied, including Batch Normalization, threshold tuning, preprocessing improvements, and longer training.

The final system was evaluated using IoU, Precision, Recall, and F1-score metrics and was capable of successfully segmenting salient objects from images.

4. Main Part

4.1. Dataset and Preprocessing

4.1.1. Dataset

This project uses the MSRA10K Salient Object Detection dataset. MSRA10K is a standard dataset for SOD tasks and contains RGB images with corresponding ground-truth saliency masks. Each image has one mask that identifies the salient foreground region.

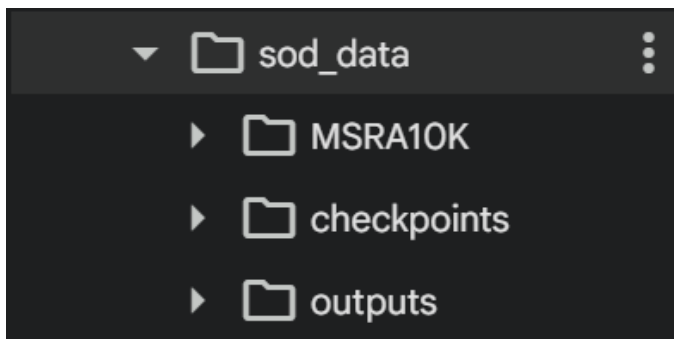


Figure 1. Mask and corresponding image from the MSRA10K dataset

The dataset source used for this project is:

<https://www.kaggle.com/datasets/evvalaycan/saliency-object-segmentation-msra10k>

The dataset was not included directly in the GitHub repository because of its size. Instead, it was stored in Google Drive using the following structure:



The images/ folder contains the RGB input images in .jpg format. The masks/ folder contains the corresponding saliency masks in .png format. Matching files share the same base filename:

101.jpg -> input RGB image

101.png -> corresponding saliency mask

Before training, the dataset was checked to ensure that every image had a matching mask.

4.1.2. Data Preprocessing

Several preprocessing steps were applied before training.

First, all images and masks were resized to:

128 x 128

This resolution was selected because it reduces GPU memory usage and allows faster training in Google Colab. Although a larger resolution could preserve more boundary details, 128 x 128 was a practical choice for a from-scratch baseline model.

RGB image values were converted from the original 0-255 range into floating-point values in the range:

[0, 1]

This normalization improves numerical stability during training and is a standard preprocessing step for neural networks.

The ground-truth masks were converted into binary format:

0 -> background

1 -> salient object

Nearest-neighbor interpolation was used when resizing masks. This is important because masks contain label values, not natural image colors. Using bilinear interpolation for masks could create intermediate gray values that do not clearly represent either background or foreground.

4.1.3. Dataset Splitting

The dataset was split into training, validation, and test sets using a 70/15/15 split.

Split	Percentage
Training	70%
Validation	15%

Test 15%

The training set was used to update the model weights. The validation set was used to monitor generalization performance and select the best prediction threshold. The test set was kept separate and used for the final evaluation.

4.1.4. Data Augmentation

Data augmentation was applied only to the training set. The goal was to reduce overfitting and improve the model's ability to generalize to unseen images.

The augmentations included:

- horizontal flipping;
- brightness variation;
- contrast variation;
- saturation variation.

Horizontal flipping was applied to both the image and the mask so that they remained aligned. Brightness, contrast, and saturation changes were applied only to the RGB image because they affect appearance but not the shape or position of the salient object.

4.2. Model Architecture

4.2.1. Baseline Model

The first version of the model was a simple encoder-decoder CNN. The encoder used convolutional layers and max-pooling to extract features from the input image. The decoder used transposed convolution layers to upsample the feature maps back to the original mask size.

The baseline followed this structure:

Input RGB image

-> Conv2D + ReLU + MaxPool

-> Conv2D + ReLU + MaxPool

-> Conv2D + ReLU + MaxPool

-> ConvTranspose2D + ReLU

-> ConvTranspose2D + ReLU

-> ConvTranspose2D + ReLU

-> 1-channel Sigmoid output

The baseline model was useful because it confirmed that the data pipeline, loss function, training loop, and evaluation process were working. However, the predicted masks were not very accurate. The main weakness was that repeated pooling reduced spatial detail, so the decoder had difficulty reconstructing precise object boundaries.

4.2.2. Improved Model

The final model is an improved encoder-decoder CNN called ImprovedSODNet. It keeps the same general encoder-decoder idea but adds Batch Normalization, skip connections, and a stronger bottleneck.

The architecture consists of:

- three encoder blocks;
- one bottleneck block;
- three decoder blocks;
- skip connections between encoder and decoder stages;
- one final 1-channel sigmoid output layer.

The simplified structure is:

Input image

-> Encoder block 1

-> Encoder block 2

-> Encoder block 3

-> Bottleneck

-> Decoder block 3 + skip connection

-> Decoder block 2 + skip connection

-> Decoder block 1 + skip connection

-> 1-channel Sigmoid mask

Batch Normalization was added after convolutional layers to improve training stability. Skip connections were added because segmentation models need spatial detail. When the encoder downsamples the image, some boundary and location information is lost. Skip connections pass earlier feature maps directly to the decoder, helping the model recover object shape and boundaries.

The model was still implemented completely from scratch and did not use pretrained models or segmentation model libraries.

4.3. Training Procedure

4.3.1. Loss Function

The model was trained using a combination of Binary Cross-Entropy and an IoU-based loss term:

$$\text{Loss} = \text{BCE} + 0.5 * (1 - \text{IoU})$$

Binary Cross-Entropy treats each pixel as a binary classification problem: background or salient object. However, BCE alone does not directly optimize the overlap between the predicted mask and ground truth. The IoU term was added to encourage better region-level overlap.

This combined loss helped the model learn both pixel-level correctness and overall mask quality.

4.3.2. Optimizer and Learning Rate

The optimizer used was Adam with a learning rate of:

1e-3

Adam was selected because it works well for neural network training and adapts updates for each parameter. A learning-rate scheduler was also used to reduce the learning rate when validation performance stopped improving.

4.3.3. Checkpointing and Early Stopping

Validation was performed after every epoch. The best model checkpoint was saved based on validation IoU. The checkpoint stored the model weights, optimizer state, epoch number, best validation IoU, and image size.

Early stopping was included to stop training when validation loss did not improve for several consecutive epochs. This helps avoid unnecessary training and reduces the risk of overfitting.

4.4. Evaluation Metrics

The model was evaluated using IoU, Precision, Recall, and F1-score.

$\text{IoU} = \text{Intersection} / \text{Union}$

$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

$\text{F1} = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$

The prediction threshold selected for the final model was 0.40 because it achieved the best validation F1-score.

4.5. Results and Improvements

The baseline model produced moderate results but struggled with object boundaries and incomplete masks. After improving the architecture and training setup, the final model produced stronger results.

Model	IoU	Precision	Recall	F1-score
-------	-----	-----------	--------	----------

Baseline	0.4262	0.6043	0.6356	0.6176
Improved	0.6470	0.7582	0.8267	0.7895

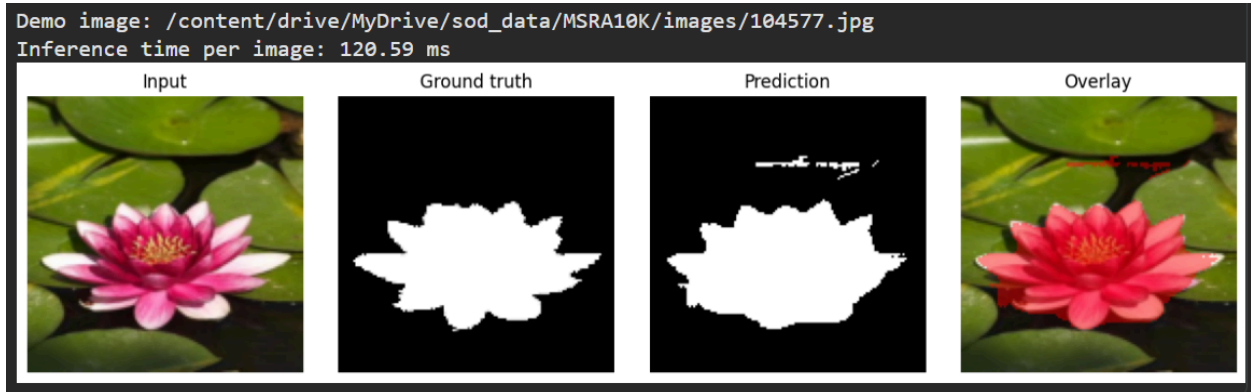


Figura 2. Prediction example

The improved model achieved a test IoU of 0.6470 and an F1-score of 0.7895. Compared with the baseline, this is a clear improvement. Precision increased, which means the model produced fewer false-positive salient regions. Recall also increased, meaning the model captured more of the true salient objects.

The most important improvements were:

- adding Batch Normalization;
- adding skip connections;
- using safer data augmentation;
- training for more epochs;
- using learning-rate scheduling;
- tuning the prediction threshold on the validation set.

These changes improved performance without using pretrained models or changing the project into a more complex framework.

5. Conclusion and Future Work

This project successfully implemented a complete Salient Object Detection pipeline from scratch using PyTorch. The project includes dataset organization, preprocessing, augmentation, model design, training, validation, evaluation, checkpointing, and demo visualization.

The final model used an improved encoder-decoder CNN with Batch Normalization and skip connections. It achieved better results than the baseline model, reaching a test IoU of 0.6470 and an F1-score of 0.7895. The project met its main goal of building a working SOD system without pretrained models.

Future work could include training at a higher resolution such as 224 x 224, adding more advanced augmentations, experimenting with dropout, or testing the model on another SOD dataset such as ECSSD or DUTS. Another possible improvement would be adding more refined decoder blocks to improve boundary quality.